

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

Error Analysis Task

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0405

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

Duration: 30 minutes

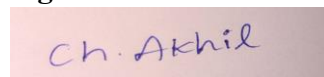
Marks: 25

Code of Conduct:

I Ch.Akhil(192525250) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



QUESTIONS:

Total Marks:25

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

- Identify the conceptual error in the likelihood formulation.
- Explain how this mistake affects parameter estimation.
- Suggest the correct mathematical formulation and reasoning.
- How would using log-likelihood fix computational issues?

Answer:**a) Conceptual Error:**

The main conceptual error is defining the likelihood function as the sum of probabilities instead of the product of probabilities. In Maximum Likelihood Estimation (MLE), each training sample is assumed to be independent, so the joint probability of all observations is obtained by multiplying their individual probabilities. Using addition violates the principle of probability and produces an incorrect objective function.

b) Effect on Parameter Estimation:

Because the likelihood is incorrectly formulated, the optimization algorithm searches for parameters that maximize the wrong function. As a result, the estimated parameters are biased and do not represent the actual data distribution. Even with a large amount of training data, the model cannot learn the true parameters, leading to poor prediction accuracy and weak generalization on unseen data.

c) Correct Mathematical Formulation:

For independent observations,

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

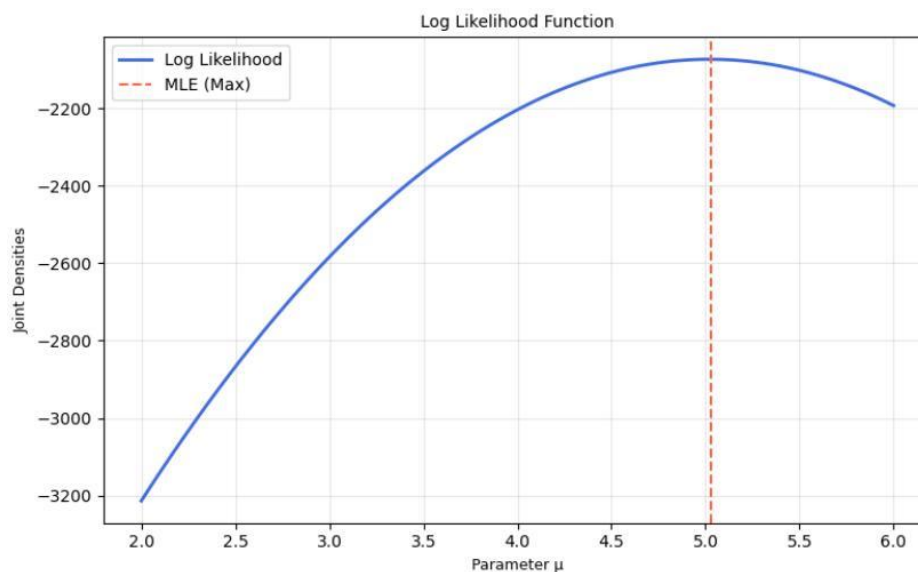
where $L(\theta)$ is the likelihood function and θ represents the model parameters. This formulation correctly represents the probability of observing the entire dataset. Maximizing this product gives the parameter values that best explain the observed data.

d) Role of Log-Likelihood:

The log-likelihood is

$$\log L(\theta) = \sum_{i=1}^n \log P(x_i | \theta)$$

It converts multiplication into addition, making differentiation and optimization much easier. It also prevents numerical underflow caused by multiplying many very small probabilities, resulting in more stable and efficient computation.



2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

- List possible implementation errors in the learning algorithm.
- Analyze how incorrect learning rate or weight updates affect convergence.
- Identify dataset-related issues that may cause this problem.
- Propose modifications to ensure convergence.

Answer:

a) Possible Implementation Errors:

The Perceptron may fail to converge due to several implementation mistakes such as an incorrect weight update rule, forgetting to include the bias term, using the wrong activation or prediction function, updating weights in the wrong direction, incorrect stopping conditions, or programming errors in the training loop. Any of these errors can prevent the decision boundary from moving correctly.

b) Effect of Learning Rate and Weight Updates:

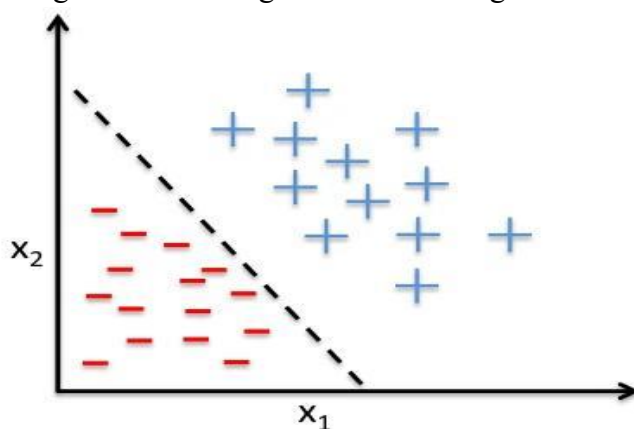
A learning rate that is too high causes the weights to overshoot the optimal solution and oscillate continuously. A very small learning rate makes convergence extremely slow. If the weight update equation is implemented incorrectly, the model may move away from the correct solution instead of toward it, causing repeated classification errors.

c) Dataset-Related Issues:

Although the dataset may be theoretically linearly separable, practical issues such as mislabeled examples, missing values, inconsistent preprocessing, or lack of feature normalization can affect convergence. If the dataset is actually not linearly separable, the Perceptron algorithm will never converge.

d) Modifications to Ensure Convergence:

Normalize all input features, include a bias term, verify data labels, use the correct Perceptron learning rule, select an appropriate learning rate, shuffle the training data after each epoch, and ensure that the dataset is linearly separable. Under these conditions, the Perceptron Convergence Theorem guarantees convergence.



Example of a linear decision boundary for binary classification.

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

- a) Identify possible mathematical or coding errors in gradient computation.
- b) Explain the role of activation functions in gradient flow.
- c) Analyze how vanishing or exploding gradients affect training.
- d) Suggest techniques to fix the issue (e.g., normalization, initialization).

Answer:

a) Possible Gradient Computation Errors:

Increasing loss during training usually indicates errors in gradient computation. Common mistakes include incorrect application of the chain rule, wrong derivative formulas for activation functions, updating weights by adding gradients instead of subtracting them, incorrect matrix dimensions, or errors in forward and backward propagation implementation. These mistakes cause the optimization process to move in the wrong direction.

b) Role of Activation Functions:

Activation functions introduce non-linearity into neural networks and determine how gradients flow backward through the network. Sigmoid and tanh functions may produce very small gradients when neurons become saturated, slowing learning. ReLU and its variants generally maintain stronger gradients and improve learning efficiency in deep networks.

c) Vanishing and Exploding Gradients:

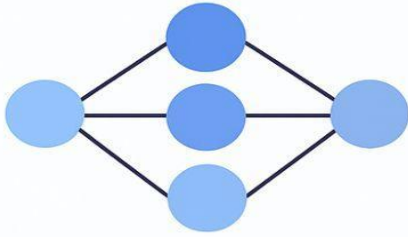
Vanishing gradients occur when gradients become extremely small, preventing early layers from learning meaningful features. Exploding gradients occur when gradients become excessively large, causing unstable weight updates and rapidly increasing loss values. Both problems make training difficult and reduce model performance.

d) Techniques to Fix the Issue:

Use Xavier or He weight initialization, normalize input data, apply Batch Normalization, use ReLU-based activation functions, perform gradient clipping, choose an appropriate learning rate, and use adaptive optimizers such as Adam. These methods improve gradient flow and stabilize the training process.

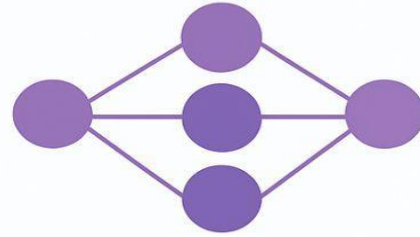
Vanishing & Exploding Gradients

Gradients can either vanish (become too small) or explode (grow uncontrollably) as they backpropagate through layers



Vanishing Gradient

Gradients shrink
towards 0



Exploding Gradient

Gradients grow
uncontrollably

4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

- Explain why batch gradient descent is slow in large datasets.
- Analyze the cause of fluctuations in SGD.
- Compare the error behavior of Mini-Batch Gradient Descent.
- Recommend the best approach and justify with error trade-offs.

Answer:

- a) Why Batch Gradient Descent is Slow:

Batch Gradient Descent computes the gradient using the entire training dataset before every weight update. For large datasets, this requires significant computational time and memory, making each iteration slow. Although the convergence path is stable, the overall training process becomes inefficient.

- b) Cause of SGD Loss Fluctuations:

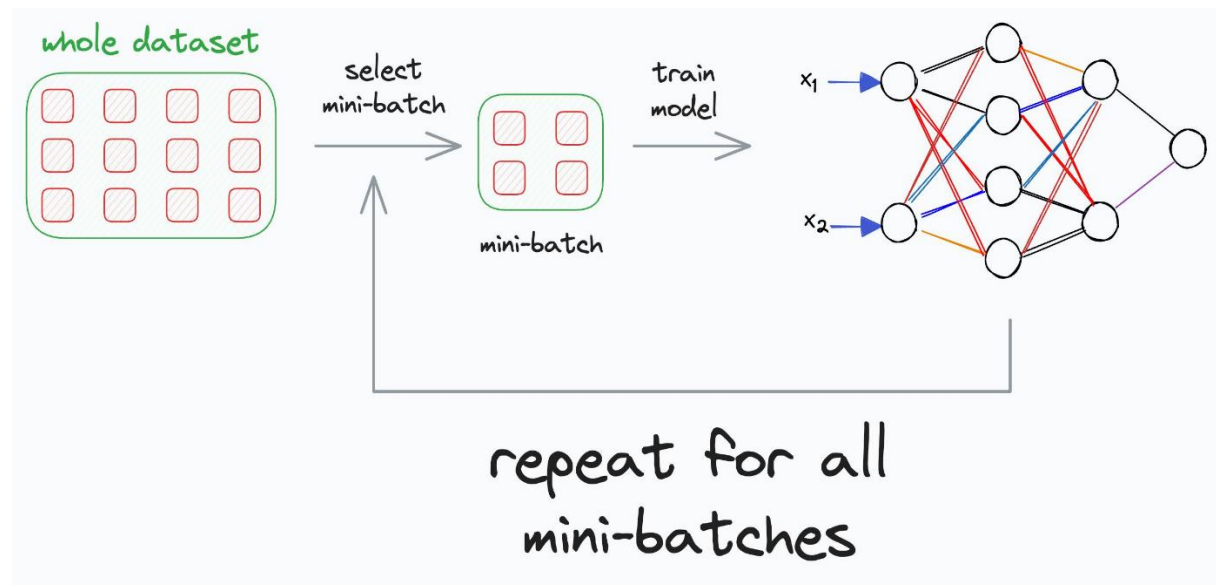
Stochastic Gradient Descent updates model parameters after processing each individual training sample. Since each sample provides only a noisy estimate of the true gradient, the optimization path fluctuates and the loss may increase or decrease between iterations. However, this randomness often helps escape shallow local minima.

- c) Mini-Batch Gradient Descent:

Mini-Batch Gradient Descent processes a small batch of samples before updating the weights. It reduces the noise seen in SGD while remaining much faster than Batch Gradient Descent. This approach provides a good balance between computational efficiency and stable convergence, making it the most widely used optimization method in deep learning.

d) Recommended Approach:

Mini-Batch Gradient Descent is generally the best choice because it combines the stability of Batch Gradient Descent with the speed of SGD. It requires less memory, converges faster, produces smoother learning curves, and works efficiently on modern GPUs, making it suitable for most machine learning applications.



5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

- Explain how high dimensionality increases model error.
- Identify signs of overfitting in this scenario.
- Analyze why distance-based learning fails in high dimensions.
- Suggest dimensionality reduction or regularization techniques to improve performance.

Answer:

a) Effect of High Dimensionality:

As the number of features increases, the feature space becomes sparse and the amount of data required to represent it grows exponentially. Models tend to memorize training examples rather than learning meaningful patterns, resulting in high variance and poor performance on unseen data. This phenomenon is known as the Curse of Dimensionality.

b) Signs of Overfitting:

The model achieves very high training accuracy but significantly lower testing accuracy. The training loss continues to decrease while the validation loss increases. Predictions on new data become inconsistent, indicating that the model has learned noise instead of useful patterns.

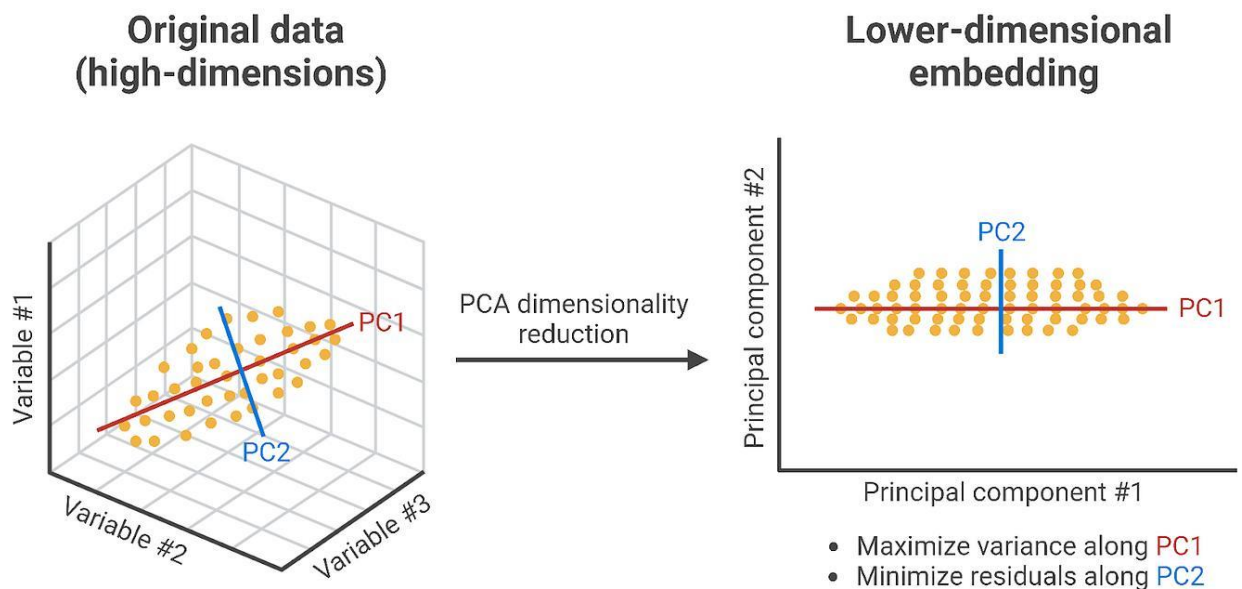
c) Why Distance-Based Learning Fails:

Algorithms such as K-Nearest Neighbors rely on distance measurements. In high-dimensional spaces, distances between different data points become very similar, reducing the ability to

distinguish between nearby and distant samples. Consequently, neighborhood-based learning becomes unreliable and classification accuracy decreases.

d) Techniques to Improve Performance:

Apply dimensionality reduction methods such as Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA), or Autoencoders to reduce unnecessary features. Use feature selection, L1/L2 regularization, dropout, cross-validation, and collect more representative training data. These techniques reduce overfitting, improve generalization, and increase model performance on unseen datasets.



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts

Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand